Lab 05: Görseller ve Sprite Temelleri

PyGame ile Oyun Geliştirme – Hafta 5

Ad Soyad: _____	Numara: _____	Tarih: _____

1 Çalışma 1: Görsel Yükleme ve Gösterme

Örnek Kod

Aşağıdaki programı `gorsel_test.py` dosyasına yazın ve çalıştırın:

```python
import os
import pygame

pygame.init()

GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Gorsel Yukleme Testi")
saat = pygame.time.Clock()

# Dosya dizini
DIZIN = os.path.dirname(os.path.abspath(__file__))
GORSEL = os.path.join(DIZIN, "assets", "images")

# Gorsel yukle
dosya_yolu = os.path.join(GORSEL, "playerShip1_blue.png")
try:
    gemi = pygame.image.load(dosya_yolu).convert_alpha()
    print("[OK] Gorsel yuklendi")
except FileNotFoundError:
    print("[UYARI] Dosya bulunamadi, placeholder oluşturuluyor")
    gemi = pygame.Surface((64, 64), pygame.SRCALPHA)
    gemi.fill((255, 0, 255, 180))

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    ekran.fill((10, 10, 40))
    ekran.blit(gemi, (GENISLIK // 2 - 32, YUKSEKLIK // 2 - 32))
    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

Görsel Dosyası Hazırlığı

`kenney.nl/assets` adresinden “Space Shooter Redux” paketini indirin. Proje klasörünüzde `assets/images/` dizini oluşturup görselleri oraya kopyalayın. Eğer indirmediyseniz program placeholder görsel oluşturacaktır.

Talep 1: Kendi Görselinizi Yükleyin

Farklı bir görsel dosyası (PNG veya JPG) yükleyin ve ekranın sol üst köşesine `(50, 50)` konumuna çizin. Terminaldeki `[OK]` mesajını gözlemleyin.

Talep 2: Birden Fazla Görsel

En az 3 farklı görsel yükleyin (gemi, düşman, arka plan) ve her birini ekranın farklı konumlarına çizin. Çizim sırasını değiştirerek z-order etkisini gözlemleyin.

Talep 3: gorsel_yukle() Fonksiyonu

Var olmayan bir dosya adı vererek `FileNotFoundError` hatasını test edin. Ardından kitaptaki `gorsel_yukle()` fonksiyonunu yazın: dosya bulunamazsa magenta renkli ve çapraz çizgili bir placeholder döndürsün.

2 Çalışma 2: convert() ve convert_alpha() Karşılaştırması

Örnek Kod

Aşağıdaki kodu `convert_test.py` dosyasına yazın:

```python
import os
import time
import pygame

pygame.init()
ekran = pygame.display.set_mode((800, 600))

DIZIN = os.path.dirname(os.path.abspath(__file__))
dosya = os.path.join(DIZIN, "assets", "images", "playerShip1_blue.png")

# 3 farkli yukleme yontemi
try:
    ham = pygame.image.load(dosya)            # Donusumsuz
    donusmus = pygame.image.load(dosya).convert()       # convert
    alpha = pygame.image.load(dosya).convert_alpha()    # convert_alpha
except FileNotFoundError:
    print("Gorsel bulunamadi, cikiliyor.")
    pygame.quit()
    exit()

# Performans testi
def blit_testi(gorsel, tekrar=10000):
    baslangic = time.time()
    for _ in range(tekrar):
        ekran.blit(gorsel, (0, 0))
    return (time.time() - baslangic) * 1000

print(f"Ham       : {blit_testi(ham):.2f} ms")
print(f"convert   : {blit_testi(donusmus):.2f} ms")
print(f"convert_a: {blit_testi(alpha):.2f} ms")

pygame.quit()
```

Talep 1: Performans Ölçümü

Kodu çalıştırın ve 3 yöntemin süresini aşağıdaki tabloya yazın:

Ham (dönüşümsüz)	
convert()	
convert_alpha()	

Talep 2: Görsel Farkı Gözlemleme

Dama deseni arkaplan üzerine hem `convert()` ile hem `convert_alpha()` ile yüklenmiş görseli yan yana çizin. Alpha kanalının korunup korunmadığını gözlemleyerek hangi yöntemin şeffaflığı koruduğunu belirleyin.

Talep 3: set_colorkey() Deneyi

`convert()` ile yüklenmiş bir görsele `set_colorkey((0, 0, 0))` uygulayın. Siyah piksellerin şeffaf olup olmadığını gözlemleyin. Ardından farklı renklerle (beyaz, kırmızı) deneyin ve sonuçları kaydedin.

3 Çalışma 3: Katmanlı Sahne Oluşturma

Örnek Kod

Aşağıdaki kodu `katmanli_sahne.py` dosyasına yazın:

```python
import os
import random
import pygame

pygame.init()
GENISLIK, YUKSEKLIK = 800, 600
ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
saat = pygame.time.Clock()

DIZIN = os.path.dirname(os.path.abspath(__file__))

def gorsel_yukle(ad, boyut=None):
    yol = os.path.join(DIZIN, "assets", "images", ad)
    try:
        g = pygame.image.load(yol).convert_alpha()
        if boyut:
            g = pygame.transform.scale(g, boyut)
        return g
    except (FileNotFoundError, pygame.error):
        yt = pygame.Surface(boyut or (64, 64), pygame.SRCALPHA)
        yt.fill((255, 0, 255, 180))
        return yt

# Yildizli arkaplan olustur
arkaplan = pygame.Surface((GENISLIK, YUKSEKLIK))
arkaplan.fill((5, 5, 25))
for _ in range(150):
    x = random.randint(0, GENISLIK)
    y = random.randint(0, YUKSEKLIK)
    arkaplan.set_at((x, y), (200, 200, 200))

# Gorseller
gemi = gorsel_yukle("playerShip1_blue.png", (64, 64))
dusman = gorsel_yukle("enemyRed1.png", (48, 48))

# 4 dusman konumu
dusmanlar = [(100 + i * 160, 100) for i in range(4)]

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    fare_x, fare_y = pygame.mouse.get_pos()

    # Z-ORDER: sira onemli!
    ekran.blit(arkaplan, (0, 0))        # 1. Arkaplan
    for dx, dy in dusmanlar:            # 2. Dusmanlar
        ekran.blit(dusman, (dx, dy))
    ekran.blit(gemi, (fare_x - 32, fare_y - 32))  # 3. Oyuncu

    pygame.display.flip()
```

Talep 1: Sahneyi Test Edin

Kodu çalıştırın. Fareyi hareket ettirerek oyuncu gemisinin düşmanların üzerinden geçtiğini doğrulayın. Çizim sırasını değiştirerek (gemiyi düşmanlardan önce çizin) ne olduğunu gözlemleyin.

Talep 2: Düşman Hareketi

Düşman gemilerini yatay olarak sağa doğru hareket ettirin. Ekranın sağ kenarından çıkan düşmanı soldan tekrar sokun (wrap stratejisi).

Talep 3: UI Katmanı Ekleyin

Ekranın sol üst köşesine bir skor metni (`pygame.font`) ve sağ üst köşesine FPS bilgisi ekleyin. Bu yazıları tüm görsellerin **üstünde** çizdiğinizden emin olun (z-order).

4 Çalışma 4: Boyut Değiştirme

Örnek Kod

Aşağıdaki programı `boyut_demo.py` dosyasına yazın ve çalıştırın:

```python
import pygame

pygame.init()
GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Boyut Degistirme")
saat = pygame.time.Clock()

# Yedek gorsel oluştur (64x64 kare)
orijinal = pygame.Surface((64, 64), pygame.SRCALPHA)
pygame.draw.polygon(orijinal, (0, 150, 255),
                    [(32, 4), (4, 60), (60, 60)])

# Farkli boyutlarda kopyalar
kucuk = pygame.transform.scale(orijinal, (32, 32))
buyuk = pygame.transform.scale(orijinal, (128, 128))
smooth = pygame.transform.smoothscale(orijinal, (128, 128))

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    ekran.fill((20, 20, 40))

    ekran.blit(kucuk, (100, 250))
    ekran.blit(orijinal, (250, 220))
    ekran.blit(buyuk, (400, 190))
    ekran.blit(smooth, (600, 190))

    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

scale() ve smoothscale() Farkı

`scale()` hızlı ama pikselleşme olabilir. `smoothscale()` daha kaliteli ama daha yavaştır. Farkı 128x128 boyutundaki iki görselde karşılaştırın.

Talep 1: Boyut Etiketleri

Her görselin altına boyutunu gösteren metin ekleyin. `pygame.font.SysFont("Arial", 16)` ile yazı tipi oluşturun. Örneğin kucuk görselin altına “32x32” yazın.

Talep 2: En-Boy Oranı Koruma

Aşağıdaki fonksiyonu programınıza ekleyin ve dikdörtgen bir görsel (100x50) oluşturup en-boy oranını koruyarak genişliği 200'e ölçeklendirin. Sonucu oranı bozulmuş bir versiyonla (200x200) yan yana gösterin.

```
def boyut_oranli(surface, hedef_genislik):
    oran = hedef_genislik / surface.get_width()
    yeni_yuk = int(surface.get_height() * oran)
    return pygame.transform.scale(
        surface, (hedef_genislik, yeni_yuk))
```

Talep 3: Dinamik Boyutlandırma

Fare tekerleğiyle (MOUSEWHEEL olayı) görselin boyutunu artırıp azaltın. `olay.y` yukarı kaydırmada +1, aşağıda -1 döner. Minimum 16, maksimum 256 piksel sınırı koyun. Her boyut değişikliğinde **orijinalden** ölçeklendirme yapın.

5 Çalışma 5: Döndürme ve Çevirme

Örnek Kod – Döndürme

Aşağıdaki programı `dondurme_demo.py` dosyasına yazın:

```python
import pygame

pygame.init()
ekran = pygame.display.set_mode((600, 400))
pygame.display.set_caption("Dondurme Demo")
saat = pygame.time.Clock()

# Ucgen seklinde gorsel
orijinal = pygame.Surface((64, 64), pygame.SRCALPHA)
pygame.draw.polygon(orijinal, (255, 200, 0),
                    [(32, 4), (4, 60), (60, 60)])
aci = 0

çalıştır = True
while çalıştır:
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    aci += 2
    if aci >= 360:
        aci -= 360

    # Orijinalden dondur, center koru
    donmus = pygame.transform.rotate(orijinal, aci)
    rect = donmus.get_rect(center=(300, 200))

    ekran.fill((20, 20, 40))
    ekran.blit(donmus, rect)
    pygame.display.flip()
    saat.tick(60)

pygame.quit()
```

Birikimli Döndürme Hatası

Orijinali saklamadan döndürme yapmak (her karede önceki sonucu döndürmek) kalite kaybına ve sürekli büyüyen bir Surface'e neden olur. Her zaman **orijinalden** döndürün.

Talep 1: Döndürme Hızı Kontrolü

Yukarı/aşağı ok tuşlarıyla döndürme hızını artırıp azaltın. `hiz` değişkeni oluşturun (başlangıç: 2). Yukarı ok ile artırın, aşağı ok ile azaltın. Hız değerini ekranda gösterin.

Talep 2: Fareye Döndürme

`math.atan2()` kullanarak görseli farenin konumuna doğru döndürün. Gemi ile fare arasına ince bir çizgi çizin. Açıyı ekranda gösterin.

```
import math
dx = fare_x - nesne_x
dy = fare_y - nesne_y
aci = math.degrees(math.atan2(-dy, dx)) - 90
```

Talep 3: Karakter Yön Değiştirme

`cevirme_demo.py` dosyası oluşturun. Sağa bakan bir üçgen çizin. `pygame.transform.flip()` ile sol yönlü versiyonunu oluşturun. Ok tuşlarıyla hareket ederken yöne göre doğru görseli gösterin. Karakterin altına yarı saydam bir yansıması ekleyin:

```
yansima = pygame.transform.flip(gorsel, False, True)
yansima.set_alpha(80)
ekran.blit(yansima, (x, y + gorsel.get_height()))
```

6 Çalışma 6: Sprite Sınıfı ve Gruplar

Örnek Kod – Temel Sprite

Aşağıdaki programı `temel_sprite.py` dosyasına yazın ve çalıştırın:

```python
import pygame
import sys

pygame.init()

GENISLIK = 800
YUKSEKLIK = 600

ekran = pygame.display.set_mode((GENISLIK, YUKSEKLIK))
pygame.display.set_caption("Temel Sprite")
saat = pygame.time.Clock()

# Basit bir Sprite sınıfı
class Kutu(pygame.sprite.Sprite):
    def __init__(self, renk, x, y, boyut=50):
        super().__init__()
        self.image = pygame.Surface((boyut, boyut), pygame.SRCALPHA)
        pygame.draw.rect(self.image, renk,
                         (0, 0, boyut, boyut), border_radius=5)
        self.rect = self.image.get_rect(center=(x, y))

    def update(self):
        pass  # Simdilik bos

# Sprite oluştur
kutu = Kutu((0, 180, 255), GENISLIK // 2, YUKSEKLIK // 2)
grup = pygame.sprite.Group(kutu)

çalıştır = True
while çalıştır:
    saat.tick(60)
    for olay in pygame.event.get():
        if olay.type == pygame.QUIT:
            çalıştır = False

    grup.update()
    ekran.fill((20, 20, 30))
    grup.draw(ekran)
    pygame.display.flip()

pygame.quit()
sys.exit()
```

Sprite ve Group

Her `Sprite` alt sınıfı `image` (görünüm) ve `rect` (konum) özelliklerine sahip olmalıdır. `Group` bu özellikleri kullanarak `draw()` ve `update()` işlemlerini toplu yapar.

Talep 1: Birden Fazla Sprite ve Hareket

Üç farklı renkte ve konumda kutu oluşturun. Hepsini aynı gruba ekleyin. `update()` metoduna ok tuşlarıyla hareket ekleme kodu yazın. Kutunun WASD veya ok tuşlarıyla dört yönde hareket etmesini sağlayın. `rect.clamp_ip()` ile ekran sınırları içinde tutun.

Talep 2: Grup İşlemleri

Aşağıdaki kodu yazın ve test edin:

```
import random

class Top(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.renk = (random.randint(100, 255),
                     random.randint(100, 255),
                     random.randint(100, 255))
        self.image = pygame.Surface((20, 20), pygame.SRCALPHA)
        pygame.draw.circle(self.image, self.renk, (10, 10), 10)
        self.rect = self.image.get_rect(
            center=(random.randint(50, 750),
                    random.randint(50, 550)))
        self.hiz_x = random.choice([-3, -2, 2, 3])
        self.hiz_y = random.choice([-3, -2, 2, 3])

    def update(self):
        self.rect.x += self.hiz_x
        self.rect.y += self.hiz_y
        if self.rect.left < 0 or self.rect.right > 800:
            self.hiz_x *= -1
        if self.rect.top < 0 or self.rect.bottom > 600:
            self.hiz_y *= -1
```

15 adet `Top` oluşturup gruba ekleyin. Space ile yeni top ekleyin, R ile rastgele bir topu `kill()` ile silin. Eleman sayısını ekranda gösterin.

Talep 3: Kayan Yıldız Arka Planı

Aşağıdaki `Yildiz` sınıfını kullanarak 50 yıldızlık bir arka plan oluşturun:

```
class Yildiz(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.boyut = random.randint(1, 3)
        cap = self.boyut * 2
        self.image = pygame.Surface((cap, cap), pygame.SRCALPHA)
        parlaklik = 100 + self.boyut * 50
        renk = (parlaklik, parlaklik, parlaklik)
        pygame.draw.circle(self.image, renk,
                           (self.boyut, self.boyut), self.boyut)
        self.rect = self.image.get_rect()
        self.rect.x = random.randint(0, 800)
        self.rect.y = random.randint(0, 600)
        self.hiz = self.boyut * 0.8 + random.uniform(0, 1)

    def update(self):
        self.rect.y += self.hiz
        if self.rect.top > 600:
            self.rect.x = random.randint(0, 800)
            self.rect.bottom = 0
```

Yıldızları `tum_spritelar` ve `yildizlar` gruplarına ekleyin. Paralaks derinlik efekti ekleyin: boyut 1 yavaş (uzak), boyut 3 hızlı (yakın).

7 Çalışma 7: Mini Proje – Uzay Sahnesi

Proje Gereksinimleri

Aşağıdaki gereksinimleri karşılayan bir **uzay sahnesi** programı yazın:

- Ok tuşları veya WASD ile kontrol edilen bir `Oyuncu` Sprite'ı
- En az 60 adet kayan `Yildiz` arka planı
- `tum_spritelar` ve `yildizlar` olmak üzere en az 2 grup
- `clamp_ip()` ile ekran sınırı kontrolü
- Sol üst köşede hayatta kalma süresi sayacı (saniye)
- Tüm güncelleme ve çizim işlemleri grup metodlarıyla yapılmalı

Talep 1: Temel Yapı

Yukarıdaki gereksinimleri karşılayan programı yazın. Oyuncu gemisi ekranın alt ortasında başlamalıdır.

Talep 2: FPS ve HUD

Sağ üst köşeye FPS değerini, sol üst köşeye hayatta kalma süresini gösteren bir bilgi paneli ekleyin. `saat.get_fps()` fonksiyonunu kullanın.

Talep 3: Egzoz Efekti

Geminin arkasından küçük turuncu/sarı daireler bırakan bir egzoz efekti ekleyin. Her karede geminin altında yeni bir `Egzoz` Sprite'ı oluşturun. Egzoz Sprite'ları birkaç kare sonra `kill()` ile kendini silsin.

Bonus: Meteor Engellerinden Kaçış

Ekranın üstünden rastgele konumlarda aşağı doğru kayan `Meteor` Sprite'ları ekleyin. Meteorlar farklı boyut ve hızlarda olsun. Henüz çarpışma algılama yapmıyoruz ama meteorların görsel olarak tehdit oluşturmasını sağlayın. Meteor grubunu ayrı tutun.

- Her 2 saniyede bir yeni meteor
- 3 farklı boyut (küçük, orta, büyük)
- Büyük meteorlar daha yavaş, küçükler hızlı
- Ekran dışına çıkan meteorlar `kill()` ile silinsin

Başarılar!